

1. Control Arm Documentation — Version 2.0

The focus of this documentation is outline how to write MoveIt2 Python code, due to the lack of information and documentation present on the internet.

This document contains oversight on how the provided Python script functions and the minimal components for robotic arm movement so that users can building their own scripts from the minimal requirements.

1.1. Python Code

1.1.1. Libraries

1. time
2. `rclpy.node`
3. JointTrajectory

```

28 #!/usr/bin/env python3
27
26 import time
25 import rclpy
24 from rclpy.node import Node
23 from trajectory_msgs.msg import JointTrajectory, JointTrajectoryPoint
22

```

Figure 1: Imported Libraries

The **rclpy** (ROS Client Library for Python) is the integrated Python library that allows for communication of the MoveIt2 (ROS2) commands to make auriculate movements of a compatible robotic arm. The library uses pre-defined classes as seen with the imported JointTrajectory and JointTrajectoryPoint libraries that greatly allow for the arm movements. The library itself is not exclusive to robotic arms but any robotic features.

Accessing the ROS2 integrated libraries users must **"source"** their environments to allow usages of ROS2 and MoveIt2 applications.

1.1.2. Node

The **class ServoNode(Node)** inheres the classes within rclpy. The functions **def __init__** and **def send** initiate the class and allow for "messages" to be sent from the node to the robotic arm.

Each joint for the Kinova arm is listed which can be articulated later when commands are sent, providing fine-tuned commands.

```

20 class JointNode(Node):
19     def __init__(self):
18         super().__init__('kinova_joint_controller')
17
16         self.publisher = self.create_publisher(JointTrajectory, '/joint_trajectory_controller/joint_trajectory', 10)
15
14         self.joints = [
13             'joint_1',
12             'joint_2',
11             'joint_3',
10             'joint_4',
9             'joint_5',
8             'joint_6',
7             'joint_7'
6         ]

```

Figure 2: Servo Class Overview

```

4   def send(self, positions, duration):
3       trajectory = JointTrajectory()
2       trajectory.joint_names = self.joints
1
29      point = JointTrajectoryPoint()
1       point.positions = positions
2       point.velocities = [0.0] * 7
3       point.time_from_start.sec = int(duration)
4       point.time_from_start.nanosec = int((duration - int(duration)) * 1e9)
5
6       trajectory.points.append(point)
7       self.publisher.publish(trajectory)
8
9       time.sleep(duration + 1.0)
10

```

Figure 3: Send Instructions to Kinova Arm

1.1.3. Main

The `rclpy.init` initialises the ROS2 library is required for the script and the ROS2 to begin undertaking commands.

```

20 def main():
19     rclpy.init()
18     node = JointNode()
17

```

Figure 4: Initialising Nodes to Send Commands

The following **Try Loop** will take the user input known as *values* using the information sending the instructions via the ROS2 node.

Once no longer required the user and press **ctrl+c** to end the input session.

The following end all node and communication sessions as to ensure no remaining messages can be sent or process remain.

```

16     try:
15         while True:
14             values = list(map(float, input('>> ').split(',')))
13
12             node.send(positions=values[:7], duration=values[7])
11
10         except KeyboardInterrupt:
9             print('Terminated Script')
8
7         finally:
6             node.destroy_node()
5             rclpy.shutdown()
4
3
2 if __name__ == '__main__':
1     main()

```

Figure 5: Input Loop and Ending Nodes When Exited